Case Study Report on

State-Based Step Tracker Using JavaScript

By [Kanishk Jitendra Singh] [150096725104]

B.Tech CSE 2025-29 HTML-5, CSS, JavaScript

Semester II

1. Introduction to the Case Study

Multi-step workflows, such as e-commerce checkout processes, are a fundamental part of modern web applications. A major challenge in these workflows is maintaining the user's progress and entered data so that they do not have to start over if they accidentally refresh the page or navigate away.

This case study focuses on designing and implementing a State-Based Step Tracker using JavaScript. The system manages a multi-step checkout workflow and remembers the current step—as well as form inputs—even after a page reload by utilizing browser storage. This project demonstrates core front-end development concepts, including state persistence, dynamic Document Object Model (DOM) manipulation, and user journey continuity without relying on a back-end database.

2. Problem Statement / Case Background (Abstract)

Background :

Traditional web forms often lose all entered data and progress if the browser is refreshed, leading to a poor user experience. Modern web applications solve this by managing "state"—a snapshot of the application's current status and data.

Abstract : This case study presents the design and implementation of a front-end Step Tracker that simulates a 4-step checkout process (Cart, Shipping Address, Payment, Confirmation). The application uses JavaScript to track step transitions using a numeric index (`currentstep`). It dynamically updates the UI to show only the active step content and updates visual step indicators. By implementing the browser's `localStorage` API, the application automatically saves the user's current step and form data in JSON format. Upon reloading, the script retrieves this stored value, parses it, and accurately restores the workflow. The project emphasizes the use of modern ES6 concepts, such as template literals and arrow functions, to handle dynamic UI transitions smoothly.

3. Case Study Design

The State-Based Step Tracker system is designed as a modular front-end application with the following core components:

1. State Initialization and Storage Integration

- **The application uses a `loadStep()` function triggered on window load to check `localStorage` for an existing step value and saved form data.**
- **If a value exists, it is parsed into a numeric state; otherwise, the application defaults to Step 1.**

2. Multi-Step Flow Design

- **A structured workflow models a 4-step process utilizing variables `let currentStep = 1;` and `const maxSteps = 4;`.**

3. Step Navigation Logic

- **`nextStep()` and `prevStep()` functions handle user progression.**
- **Boundary checks (e.g., `if (currentStep < maxSteps)`) ensure the state cannot increment past step 4 or decrement below step 1.**

4. Dynamic UI Step Rendering

- **A centralized `renderStep()` function handles DOM manipulation. It iterates through a loop to remove the `active` class from all `.step` and `.step-content` elements, then selectively adds**

the `active` class back only to the elements matching the `currentStep`.
- A visual indicator dynamically displays text like "Step 2 of 4".

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

Key Methods and Logic:

1. State Management: Utilizing a numeric variable (`currentStep`) to track sequential progress.
2. Data Persistence: Using `localStorage.setItem('currentStep', ...)` to write state data and `localStorage.getItem()` to read it.
3. Data Serialization: Implementing `JSON.stringify()` to save nested form objects and `JSON.parse()` to restore them upon reload.
4. Event Listening: Attaching the `saveStep` function to the document's `input` event to capture and persist typing in real-time.

Technology Stack Used for the Case

- Languages: HTML5, CSS3 (with a mobile-first responsive approach), and JavaScript.

- **ES6 Concepts Applied:**
  - **Arrow functions for clean event listener syntax (`window.onload = () => loadStep();`).**
  - **Template literals for dynamic class targeting and text updates (e.g., `document.getElementById(`step-${currentStep}`)`).**
  - **Optional chaining (`?.`) to safely read input values.**
- **Environment: Code editor and modern web browser.**

5. Problem Statement / Case Study Implementation Details and Snapshots.

The implementation relies heavily on vanilla JavaScript for logic and CSS for visual toggling. Key implementation details from the source code include:

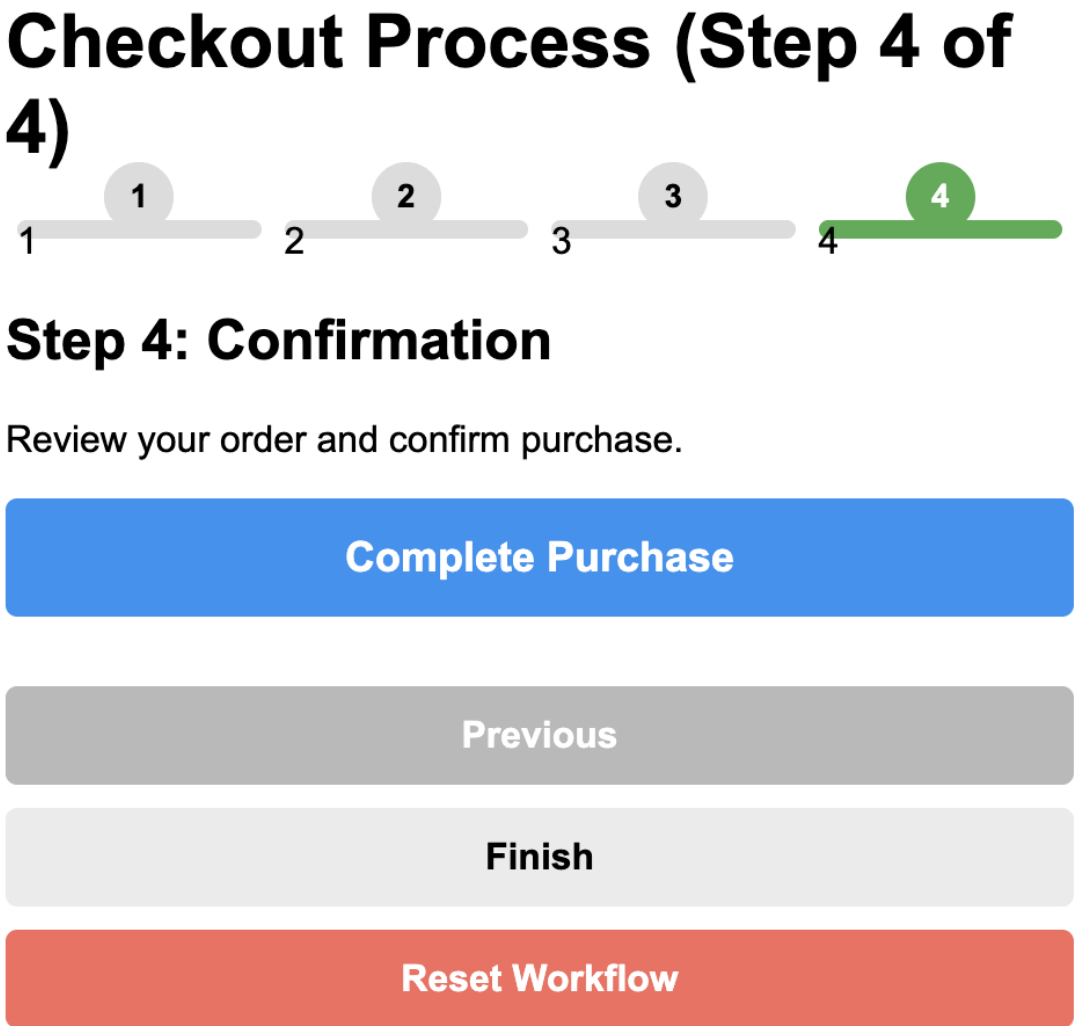
1. Restoring State on Reload: The application initializes by accurately restoring the step using ES6 Number parsing:

2. Step Navigation & Boundary Checks: Navigation strictly checks boundaries to prevent rendering errors before saving the new state:

3. Form Data Persistence: To ensure form data is not lost, an object is constructed and saved using JSON parsing:

4. Dynamic UI Switching: The rendering function manages visibility without page reloads by clearing old states and applying the `active` class:

5. Reset Workflow: A dedicated button clears the entire storage and forces the UI back to Step 1:



6. Problem Statement / Case Study Results and Conclusion.

Findings

- **The integration of `localStorage.setItem()` and `getItem()` successfully prevents the loss of user progress and form inputs during accidental page reloads.**

- Using `JSON.stringify` allows complex, multi-step form data to be stored securely in a single storage key (`formData`).
- Centralizing the UI updates into a single `renderStep()` function keeps the code modular and easy to debug, ensuring the visual indicators and form content are always perfectly synchronized.

Conclusion This case study successfully demonstrates the complete implementation of a state-persistent, multi-step workflow. By leveraging modern ES6 JavaScript concepts, DOM manipulation, and browser storage APIs, the project fulfills all assignment requirements, ensuring consistent workflow persistence and dynamic UI updates without full page reloads.

7. References

- MDN Web Docs - Window.localStorage API
- MDN Web Docs - Document Object Model (DOM) Manipulation
- MDN Web Docs - JavaScript ES6 Arrow Functions, Template Literals, and JSON Parsing